Práctica 03 - Auditoría de equipos con Nmap

Análisis de puertos, servicios y superficie de exposición en Ubuntu

Módulo: Seguridad y Alta Disponibilidad

Alumno: Pedro Hosannah

Curso: 2º ASIR - 2026/2027

Fecha: 16-21 de septiembre de 2026

1. Introducción

En esta práctica he utilizado Nmap para auditar los servicios expuestos por mi propia máquina virtual Ubuntu. El ejercicio plantea el trabajo desde el punto de vista de un auditor de seguridad: primero se identifican los puertos abiertos y después se analiza qué servicio existe detrás de cada uno y si realmente es necesario mantenerlo accesible.

He realizado todas las pruebas sobre localhost, es decir, sobre la propia máquina virtual, respetando la regla de seguridad indicada por el profesor: no realizar escaneos contra equipos, servidores o direcciones IP de terceros. El entorno utilizado ha sido la misma máquina virtual Ubuntu de las prácticas anteriores, ya que el enunciado proporciona los comandos directamente para Linux/Ubuntu y establece este sistema como entorno de trabajo.

2. Objetivo de la práctica

El objetivo ha sido comprobar qué puertos TCP están abiertos en la máquina virtual, identificar los servicios y versiones que responden en ellos, localizar posibles puertos que no aparecen en un escaneo básico y relacionar los resultados con los servicios publicados por Docker. Finalmente, he valorado qué servicios deberían mantenerse accesibles y qué medidas ayudarían a reducir la superficie de exposición del sistema.

3. Instalación y comprobación de Nmap

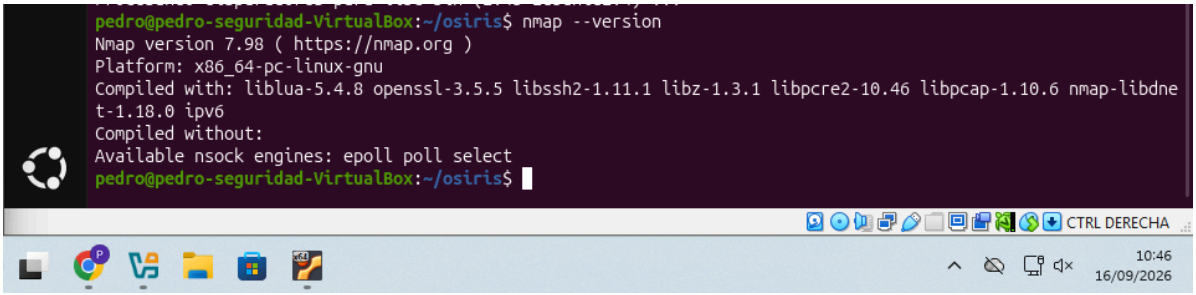
Para comenzar instalé Nmap utilizando la orden indicada por el profesor:

sudo apt install -y nmap

Después comprobé que la herramienta estaba correctamente instalada mediante:

nmap --version

La máquina devolvió Nmap 7.98, confirmando que la herramienta estaba disponible y preparada para realizar los escaneos de la práctica.



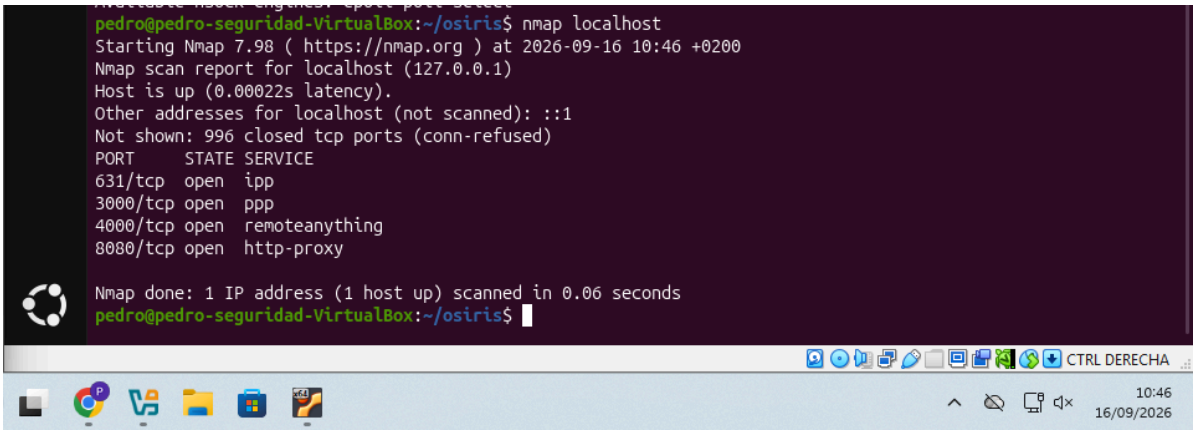
Comprobación de la instalación de Nmap y versión 7.98 en la máquina virtual Ubuntu.

4. Misión 1 - Descubre las puertas abiertas

El primer análisis consistió en escanear localhost para conocer qué puertos habituales estaban abiertos en mi propia máquina virtual. El comando utilizado fue:

nmap localhost

El resultado mostró cuatro puertos TCP abiertos: 631, 3000, 4000 y 8080. También indicó que 996 puertos TCP estaban cerrados. Un puerto abierto no significa por sí mismo que exista una vulnerabilidad; indica que hay un servicio escuchando y preparado para recibir conexiones.



Primer escaneo de localhost con Nmap, en el que se detectan los puertos 631, 3000, 4000 y 8080 abiertos.

5. Misión 2 - ¿Quién está detrás de cada puerta?

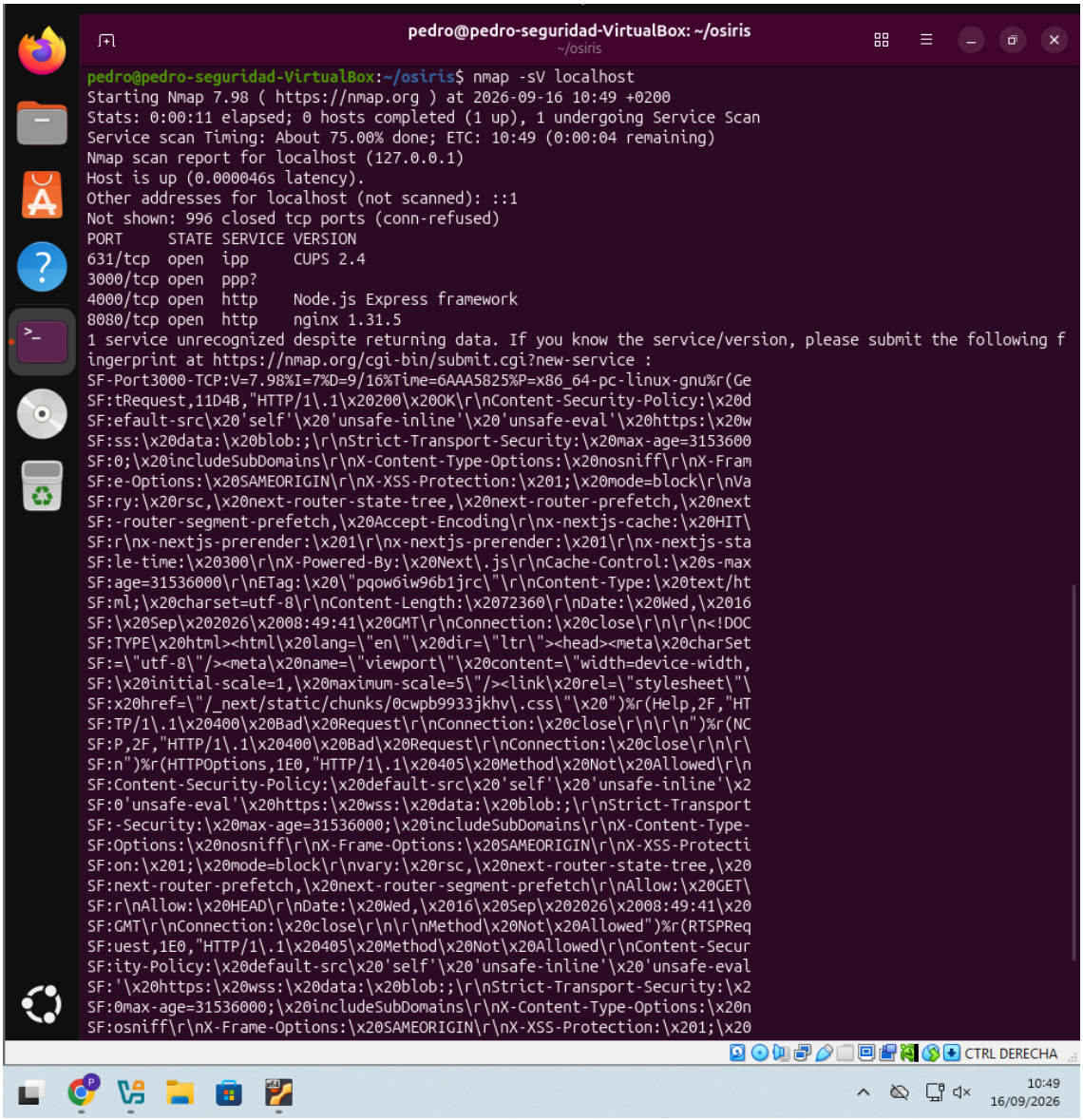
Para obtener más información utilicé la opción -sV, que intenta identificar el servicio real que está respondiendo en cada puerto y, cuando es posible, también su versión:

nmap -sV localhost

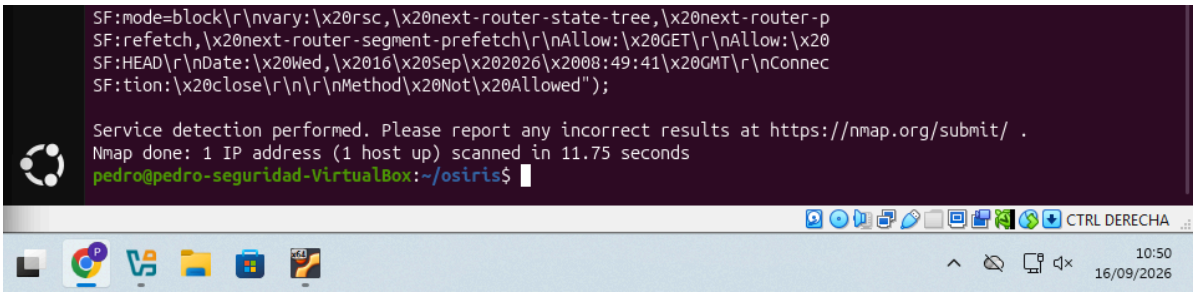
La detección de servicios permitió obtener información adicional respecto al escaneo básico. El puerto 631 fue identificado como IPP con CUPS 2.4; el puerto 4000 como HTTP con un framework Node.js Express; y el puerto 8080 como HTTP con nginx 1.31.5. En el puerto 3000 Nmap recibió respuesta, pero no pudo reconocer con certeza el servicio y lo mostró como ppp. En nuestro entorno este puerto corresponde a la interfaz web de OSIRIS, publicada mediante Docker.

¿Qué información adicional proporciona -sV?

Permite identificar con mayor precisión el servicio que escucha en cada puerto abierto y, cuando Nmap puede reconocerlo, también su versión. Esto aporta más información que el escaneo básico, que principalmente relaciona números de puerto con servicios habituales.



Escaneo de detección de servicios con nmap -sV localhost, mostrando CUPS 2.4, Node.js Express y nginx 1.31.5.



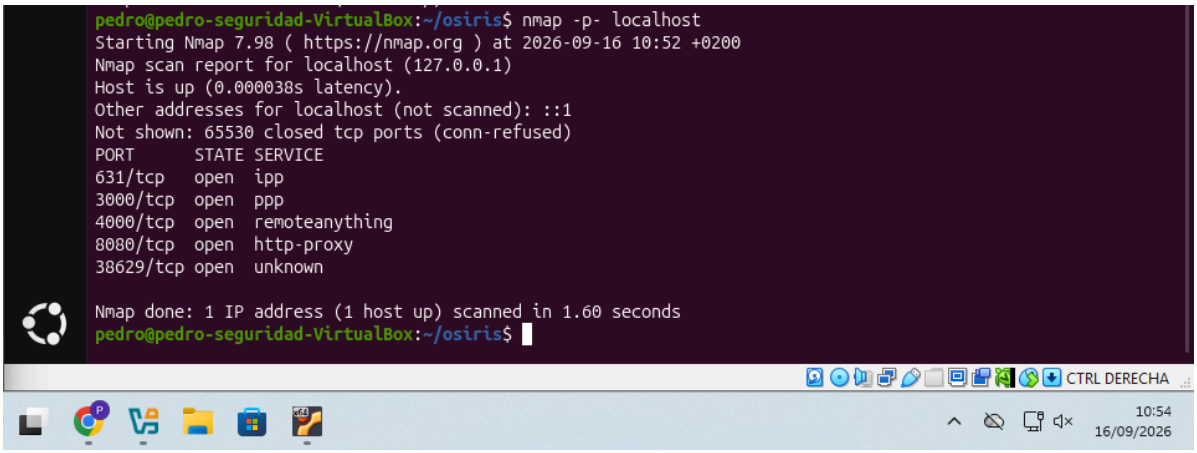
Finalización del escaneo de servicios realizado con la opción -sV.

6. Misión 3 - Busca puertas que normalmente no vemos

A continuación realicé un escaneo de todos los puertos TCP, desde el 1 hasta el 65535, utilizando:

nmap -p- localhost

El escaneo completo encontró los mismos cuatro puertos anteriores y un puerto adicional: 38629/tcp. Este puerto no había aparecido en el escaneo básico, por lo que fue necesario investigarlo de forma específica.



Escaneo completo de los 65.535 puertos TCP, en el que aparece el puerto adicional 38629/tcp.

Para intentar identificar el servicio del puerto 38629 ejecuté:

nmap -sV -p 38629 localhost

Nmap identificó un servicio HTTP basado en Golang net/http. A continuación comprobé qué proceso del sistema estaba escuchando realmente en ese puerto mediante:

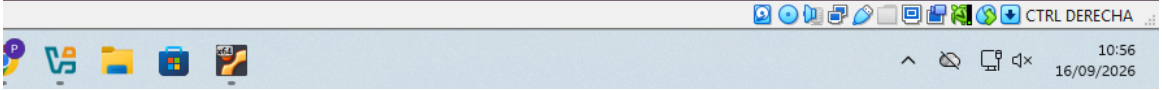
sudo ss -ltnp | grep ':38629'

La salida mostró que el proceso asociado era containerd y que estaba escuchando en 127.0.0.1:38629. Por tanto, el servicio solo estaba vinculado a la interfaz local de la máquina y no directamente a todas las interfaces de red. Su presencia está relacionada con el entorno de contenedores utilizado durante las prácticas.

```
pedro@pedro-seguridad-VirtualBox:~/osiris$ nmap -sV -p 38629 localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-16 10:55 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000025s latency).
Other addresses for localhost (not scanned): ::1

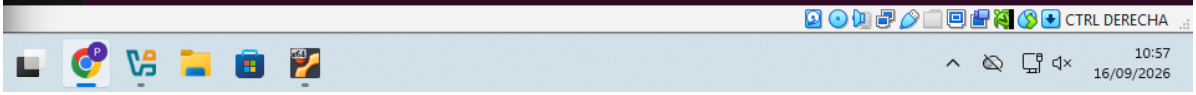
PORT      STATE SERVICE VERSION
38629/tcp open  http    Golang net/http server
1 service unrecognized despite returning data. If you know the service/version, please submit the following f
ingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port38629-TCP:V=7.98%I=7%D=9/16%Time=6AAA598B%P=x86_64-pc-linux-gnu%r(G
SF:enericLines,67,"HTTP/1\.1\x20400\x20Bad\x20Request\r\nContent-Type:\x20
SF:text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\
SF:x20Request")%r(GetRequest,8F,"HTTP/1\.0\x20404\x20Not\x20Found\r\nDate:
SF:\x20Wed,\x2016\x20Sep\x202026\x2008:55:39\x20GMT\r\nContent-Length:\x20
SF:19\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\n\r\n404:\x20Page
SF:\x20Not\x20Found")%r(HTTPOptions,8F,"HTTP/1\.0\x20404\x20Not\x20Found\r
SF:\nDate:\x20Wed,\x2016\x20Sep\x202026\x2008:55:39\x20GMT\r\nContent-Leng
SF:th:\x2019\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\n\r\n404:\
SF:x20Page\x20Not\x20Found")%r(RTSPRequest,67,"HTTP/1\.1\x20400\x20Bad\x20
SF:Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:
SF:\x20close\r\n\r\n400\x20Bad\x20Request")%r(Help,67,"HTTP/1\.1\x20400\x2
SF:0Bad\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nCon
SF:nection:\x20close\r\n\r\n400\x20Bad\x20Request")%r(SSLSessionReq,67,"HT
SF:TP/1\.1\x20400\x20Bad\x20Request\r\nContent-Type:\x20text/plain;\x20cha
SF:rset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x20Request")%r(Fou
SF:rOhFourRequest,8F,"HTTP/1\.0\x20404\x20Not\x20Found\r\nDate:\x20Wed,\x2
SF:016\x20Sep\x202026\x2008:55:54\x20GMT\r\nContent-Length:\x2019\r\nConte
SF:nt-Type:\x20text/plain;\x20charset=utf-8\r\n\r\n404:\x20Page\x20Not\x20
SF:Found")%r(LPDString,67,"HTTP/1\.1\x20400\x20Bad\x20Request\r\nContent-T
SF:ype:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400
SF:\x20Bad\x20Request")%r(SIPOptions,67,"HTTP/1\.1\x20400\x20Bad\x20Reques
SF:t\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20cl
SF:ose\r\n\r\n400\x20Bad\x20Request")%r(Socks5,67,"HTTP/1\.1\x20400\x20Bad
SF:\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnect
SF:ion:\x20close\r\n\r\n400\x20Bad\x20Request")%r(OfficeScan,A3,"HTTP/1\.1
SF:\x20400\x20Bad\x20Request:\x20missing\x20required\x20Host\x20header\r\n
SF:Content-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r
SF:\n\r\n400\x20Bad\x20Request:\x20missing\x20required\x20Host\x20header");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.00 seconds
pedro@pedro-seguridad-VirtualBox:~/osiris$
```



Análisis específico del puerto 38629, identificado por Nmap como un servicio HTTP basado en Golang net/http.

```
pedro@pedro-seguridad-VirtualBox:~/osiris$ sudo ss -ltnp | grep ':38629'
LISTEN 0      4096       127.0.0.1:38629      0.0.0.0:*    users:(("containerd",pid=1261,fd=16))
pedro@pedro-seguridad-VirtualBox:~/osiris$
```



Comprobación del proceso que escucha en 127.0.0.1:38629, asociado a containerd.

7. Misión 4 - El misterio del puerto abierto

En esta parte el profesor plantea un caso teórico en el que aparece el resultado 22/tcp open ssh. Este puerto no apareció abierto durante los escaneos realizados en mi máquina virtual, por lo que lo analizo únicamente como supuesto práctico para interpretar qué implicaciones tendría encontrar un servicio SSH expuesto.

1. ¿Qué es SSH?

SSH (Secure Shell) es un protocolo que permite conectarse de forma remota y segura a otro equipo. La comunicación se realiza cifrada, evitando que las credenciales y los datos viajen en texto plano.

2. ¿Para qué se utiliza?

Se utiliza principalmente para administrar servidores y equipos Linux de forma remota, abrir una terminal, ejecutar comandos, gestionar archivos y realizar tareas de mantenimiento sin estar físicamente delante del sistema.

3. ¿Por qué puede resultar útil?

Permite administrar sistemas a distancia de forma segura, facilitando tareas como mantenimiento, instalación de actualizaciones, revisión de configuraciones y resolución de incidencias.

4. ¿Por qué puede representar un riesgo si se expone innecesariamente?

Porque aumenta la superficie de ataque. Si el servicio está accesible sin necesidad, un atacante podría intentar realizar fuerza bruta contra las credenciales o aprovechar una vulnerabilidad de una versión mal configurada o desactualizada.

5. ¿Qué medidas podríamos aplicar para protegerlo?

Desactivar SSH cuando no sea necesario, limitar el acceso mediante firewall a direcciones autorizadas, utilizar autenticación mediante claves, impedir el acceso directo de root, emplear credenciales robustas y mantener el servicio actualizado.

Por tanto, un puerto SSH abierto puede ser legítimo y muy útil para la administración remota, pero solo debería mantenerse accesible cuando exista una necesidad real y con las medidas de seguridad adecuadas.

8. Misión 5 - Conecta lo aprendido con Docker

Para relacionar los resultados de Nmap con el despliegue de OSIRIS comprobé los contenedores en ejecución y los puertos publicados utilizando:

sudo docker ps

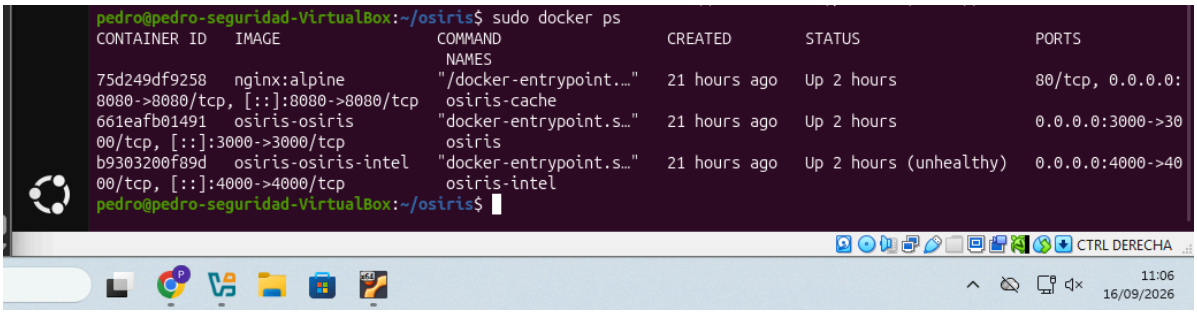
La salida mostró que el contenedor principal osiris publica el puerto 3000, osiris-intel publica el puerto 4000 y osiris-cache publica el puerto 8080. Estos tres puertos coinciden con los puertos que Nmap había detectado abiertos anteriormente.

¿Por qué Docker necesita publicar un puerto para que podamos acceder a OSIRIS desde el navegador?

Los contenedores funcionan en una red aislada. Para acceder desde la máquina virtual a un servicio que se ejecuta dentro de un contenedor, Docker debe establecer una correspondencia entre un puerto del sistema anfitrión y el puerto interno del contenedor. En OSIRIS, el puerto 3000 permite acceder desde el navegador mediante http://localhost:3000.

¿Podríamos tener un servicio funcionando dentro de un contenedor pero que no fuese accesible desde fuera del contenedor?

Sí. Un servicio puede funcionar dentro de un contenedor sin publicar su puerto hacia la máquina anfitriona. En ese caso podría ser utilizado internamente por otros contenedores de la misma red de Docker, pero no sería accesible directamente desde fuera. Esto puede ayudar a reducir la superficie de exposición cuando un servicio no necesita ser público.



Comprobación de los contenedores de OSIRIS y de los puertos 3000, 4000 y 8080 publicados mediante Docker.

En la captura también se observa que osiris-intel aparece con estado unhealthy. Este estado no modifica la identificación del puerto publicado, pero evita afirmar que todos los servicios se encuentran en estado saludable.

9. Reto final - Informe del auditor

Con los resultados obtenidos elaboré la siguiente ficha de auditoría. El puerto 22/SSH se incluye únicamente como caso teórico planteado por el profesor; no fue detectado como abierto en mi máquina virtual.

Puerto	Estado	Servicio	¿Para qué sirve?	¿Por qué está abierto?	¿Debería mantenerse abierto?
22/tcp	Caso teórico	SSH	Administración remota segura mediante una conexión cifrada.	En el supuesto planteado, existiría un servidor SSH activo para administración remota.	Solo si es necesario y protegido con claves, firewall y una configuración segura.
631/tcp	Abierto	CUPS 2.4 / IPP	Servicio de impresión mediante IPP.	El servicio CUPS está activo en Ubuntu.	Solo si se necesita impresión; si no, convendría deshabilitarlo.
3000/tcp	Abierto	OSIRIS / servicio web	Acceso a la interfaz web principal de OSIRIS.	Docker publica el puerto 3000 del contenedor osiris.	Sí durante la práctica; después solo si OSIRIS sigue siendo necesario.
4000/tcp	Abierto	HTTP / Node.js Express	Servicio asociado a osiris-intel.	Docker publica el puerto 4000 del contenedor osiris-intel.	Solo si es necesario para el entorno; evitar exponerlo sin necesidad.
8080/tcp	Abierto	HTTP / nginx 1.31.5	Servicio web utilizado por osiris-cache.	Docker publica el puerto 8080 del contenedor nginx.	Solo mientras sea necesario para el funcionamiento del entorno.
38629/tcp	Abierto	Golang net/http / containerd	Servicio local relacionado con la gestión de contenedores.	containerd escucha en 127.0.0.1:38629.	Puede mantenerse al estar limitado a localhost; no debería exponerse a otras interfaces sin necesidad.

10. Conclusión

Durante la auditoría encontré cinco puertos TCP abiertos en mi máquina: 631, 3000, 4000, 8080 y 38629. Mediante Nmap pude identificar servicios como CUPS 2.4, Node.js Express y nginx 1.31.5, además de relacionar los puertos 3000, 4000 y 8080 con los contenedores de OSIRIS. El puerto 38629 fue el más interesante porque no apareció en el escaneo básico y fue necesario realizar un escaneo completo para detectarlo. Después comprobé que estaba relacionado con containerd y que solo escuchaba en 127.0.0.1. También analicé de forma teórica el puerto 22/SSH, aunque este no apareció abierto en mi equipo. Para reducir la superficie de exposición mantendría únicamente los servicios necesarios, evitaría publicar puertos internos de Docker sin necesidad y limitaría mediante firewall cualquier servicio que deba ser accesible desde la red. En el caso de servicios de administración remota como SSH, aplicaría autenticación mediante claves y una configuración restrictiva.